

Project Report
On
Library Management System



Guide By:
Mr. Tanishk

Submitted By:
Radhika Chaurasiya

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teachers and all those who supported and guided me during the completion of my project entitled “Library Management System”.

This project has provided me with an opportunity to improve my knowledge and practical skills in Python programming, database management, and graphical user interface development. I am thankful to my teachers for their valuable guidance, encouragement, and support throughout the development of this project.

I would also like to thank my friends and family members for their motivation and support during the completion of this project.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this project.

Name: Radhika Chaurasiya

Branch: Information Technology

ABSTRACT

The Library Management System is a desktop-based application developed to simplify and automate the basic activities of a library. The main purpose of this project is to manage books, library members, book issuing, and book returning operations in an organized and efficient manner.

The system has been developed using Python programming language. The Graphical User Interface (GUI) has been created using the Tkinter library, which provides an easy and user-friendly interface for interacting with the application. SQLite has been used as the backend database to store and manage information related to books, members, and book transactions.

The system provides various features such as Add Book, View Books, Add Member, View Members, Issue Book, View Issued Books, Return Book, and View Returned Books. These features allow the user to perform library operations easily through different buttons and screens.

The project follows a simple and structured approach where the user interacts with the frontend interface, the Python program processes the information in the backend, and the SQLite database stores the required records.

This project demonstrates the practical implementation of Python programming, GUI development, database management, and CRUD operations. The Library Management System reduces manual work and helps in maintaining library records accurately and efficiently.

TABLE OF CONTENTS

S. No.	CONTENTS	PAGE NO.
1.	Acknowledgement	i
2.	Abstract	ii
3.	Table of Contents	iii
4.	Introduction	1
5.	Objectives	2
6.	Hardware and Software Requirements	3
7.	Technologies Used	4
8.	System Architecture	5
9.	User Interface Design	7
10.	Frontend Design	8
11.	Backend Design	8
12.	Database Design	9
13.	Methodology	10
14.	Modules of the System	11
15.	Working of the System	12
16.	Conclusion	17
17.	Future Scope	18
18.	Bibliography	19

INTRODUCTION

A library is an important place for collecting, storing, and providing access to books and other educational resources. In a traditional library, maintaining records of books, members, issued books, and returned books manually can be time-consuming and may lead to errors. To overcome these problems, a computerized Library Management System can be used.

The Library Management System is a desktop-based application developed to manage the basic activities of a library in a simple and organized manner. The system allows the librarian or authorized user to add books, view available books, add members, view member records, issue books, view issued books, return books, and view returned books.

This project is developed using Python as the main programming language. The graphical user interface (GUI) is designed using Tkinter, which provides different interface components such as buttons, labels, entry fields, frames, forms, and tables. SQLite is used as the database for storing and managing the records of books, members, and book transactions.

The system consists of three major parts: frontend, backend, and database. The frontend provides an easy-to-use interface through which the user interacts with the application. The backend, developed in Python, handles the application logic and processes user requests. The SQLite database stores the information permanently in the `library.db` file.

The application also provides a dashboard that displays important information such as the total number of books, total members, issued books, and returned books. This helps the user quickly understand the current status of the library.

The main purpose of developing this system is to reduce manual work, improve accuracy, save time, and provide an organized method for maintaining library records. The project also demonstrates the practical implementation of Python programming, GUI development, database connectivity, and CRUD operations.

Overall, the Library Management System provides a simple, efficient, and user-friendly solution for managing common library activities.

OBJECTIVES

The main objectives of the Library Management System are:

1. To develop a computerized system for managing library records.
2. To provide a simple and user-friendly graphical interface.
3. To add and store information about books.
4. To view all available book records.
5. To add and manage library member information.
6. To view all registered members.
7. To issue books to registered library members.
8. To maintain records of issued books.
9. To manage the return of books.
10. To maintain records of returned books.
11. To automatically update the quantity of books when a book is issued or returned.
12. To reduce manual paperwork and improve record management.
13. To use a database for secure and organized storage of information.
14. To demonstrate the practical use of Python, Tkinter, and SQLite.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

The following hardware requirements are sufficient for running the Library Management System:

Processor: Intel Core i3 or equivalent

RAM: Minimum 4 GB

Hard Disk Space: Minimum 500 MB free space

Display: Standard monitor

Keyboard and Mouse: Required for user input and navigation

Software Requirements

The following software tools and technologies are used for the development of the project:

Operating System: Windows 10 or Windows 11

Programming Language: Python

Python Version: Python 3.x

IDE: Spyder / Anaconda

Frontend Technology: Tkinter

Backend Technology: Python

Database: SQLite

Database File: library.db

TECHNOLOGIES USED

The Library Management System has been developed using the following technologies:

Python:

Python is the main programming language used to develop the application. It is responsible for handling the program logic and connecting the user interface with the database.

Tkinter:

Tkinter is Python's built-in library used to create the Graphical User Interface. It is used to create windows, buttons, labels, forms, text fields, tables, frames, and navigation menus.

SQLite:

SQLite is a lightweight database management system used to store the information of books, members, issued books, and returned books. The database is automatically stored in the file named library.db.

Spyder:

Spyder is the Integrated Development Environment used to write, edit, and run the Python program.

System Architecture Flow

USER

|



FRONTEND / USER INTERFACE

(Tkinter GUI)

|



BACKEND / APPLICATION LOGIC

(Python)

|



DATABASE

(SQLite - library.db)

SYSTEM ARCHITECTURE

The Library Management System follows a simple and organized architecture consisting of three main layers: Frontend, Backend, and Database. These three layers work together to perform different library operations efficiently.

Frontend Layer

The frontend is the visible part of the application through which the user interacts with the system. It is developed using Python Tkinter.

The frontend contains the main dashboard, navigation sidebar, buttons, forms, input fields, labels, and tables. It allows the user to perform operations such as adding books, viewing books, adding members, issuing books, and returning books.

Backend Layer

The backend is responsible for handling the main logic of the application. It is developed using Python.

The backend receives the information entered by the user, validates the data, performs the required operation, and communicates with the database.

For example, when a user adds a new book, the backend receives the book details and stores them in the database. When a book is issued, the backend checks its availability and updates the book quantity.

Database Layer

The database layer is responsible for storing and managing all the information used by the system. SQLite is used as the database in this project.

The database contains tables for books, members, and issue/return transactions. The database is stored locally in a file named `library.db`.

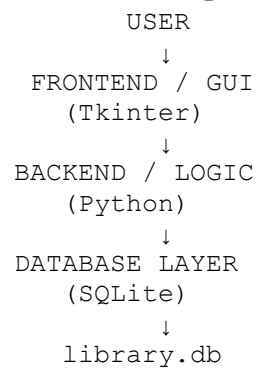
Working Flow

The overall working of the system can be represented as:

User → Frontend → Python Backend → SQLite Database

When the user performs an operation through the frontend, the request is sent to the Python backend. The backend processes the request and communicates with the SQLite database. The result is then displayed back to the user through the frontend.

System Architecture Diagram



This architecture provides a clear separation between the user interface, application logic, and data storage. It makes the system easier to understand, maintain, and modify in the future.

USER INTERFACE DESIGN

The user interface of the Library Management System has been designed to provide a simple, clean, and user-friendly experience.

The main application window contains a professional header displaying the title “Library Management System”. Below the header, the application is divided into two main sections: a sidebar navigation menu and a main content area.

The sidebar contains buttons for accessing different modules of the application. These buttons include Dashboard, Add Book, View Books, Add Member, View Members, Issue Book, View Issued Books, Return Book, and View Returned Books.

The main content area changes according to the option selected by the user. For example, when the user clicks Add Book, a form appears where the user can enter the Book ID, Book Title, Author, Category, and Quantity.

The dashboard provides a summary of the library system. It displays the total number of books, total members, issued books, and returned books. This allows the user to quickly understand the current status of the library.

The interface has been designed using Tkinter widgets such as Labels, Buttons, Entry fields, Frames, and Treeview tables.

Main User Interface Components

Component	Purpose
Header	Displays the project title
Sidebar	Provides navigation to different modules
Dashboard	Shows summary and statistics
Buttons	Perform different operations
Entry Fields	Accept user input
Forms	Collect book and member information
Treeview Table	Displays database records
Message Box	Shows success, warning, and error messages

FRONTEND DESIGN

The frontend of the Library Management System is developed using the Tkinter library in Python.

The frontend is responsible for providing an interactive environment through which the user can communicate with the system. Instead of using a command-line interface, the project uses a Graphical User Interface.

The main frontend consists of a header, sidebar navigation menu, dashboard, forms, buttons, and tables.

The header displays the title of the project and provides a professional appearance. The sidebar contains all the major navigation buttons, allowing the user to easily switch between different functions.

Forms are used for collecting information from the user. For example, the Add Book form collects information such as Book ID, Book Title, Author, Category, and Quantity.

Treeview tables are used to display information stored in the database. These tables are used in the View Books, View Members, View Issued Books, and View Returned Books modules.

The frontend has been designed to be simple and easy to understand so that even a new user can operate the system without difficulty.

BACKEND DESIGN

The backend of the Library Management System is developed using Python.

The backend is responsible for processing user input, applying program logic, connecting with the database, and managing all library operations.

Different Python functions are created for each major feature of the system. These functions perform operations such as adding books, viewing books, adding members, issuing books, and returning books.

The backend also performs validation. For example, Book ID and Member ID must contain valid numerical values. The system also checks whether a book exists before issuing it and verifies whether the requested member is registered.

When a book is issued, the backend decreases the available quantity of that book by one. When the same book is returned, the quantity is automatically increased.

The backend communicates with the SQLite database using the sqlite3 module. SQL queries are used to insert, retrieve, and update records.

DATABASE DESIGN

SQLite has been used as the database management system for this project. The database is stored in a file named library.db.

- The project contains three main tables: Books, Members, and Issues.
- The Books table stores information related to library books.
- The Members table stores information related to library members.
- The Issues table stores information about book issue and return transactions.

Books Table

Field Name	Data Type	Description
book_id	INTEGER	Unique ID of the book
title	TEXT	Name of the book
author	TEXT	Author name
category	TEXT	Book category
quantity	INTEGER	Available quantity

Members Table

Field Name	Data Type	Description
member_id	INTEGER	Unique ID of the member
name	TEXT	Name of member
course	TEXT	Course or department
contact	TEXT	Contact number

Issues Table

Field Name	Data Type	Description
issue_id	INTEGER	Unique transaction ID
book_id	INTEGER	ID of issued book
member_id	INTEGER	ID of member
issue_date	TEXT	Date of issue
return_date	TEXT	Date of return

METHODOLOGY

The development of the Library Management System follows a systematic methodology.

Step 1: Requirement Analysis

The requirements of the project were identified. The main requirements included book management, member management, book issuing, book returning, and record viewing.

Step 2: System Design

The structure of the application was designed. A frontend interface, backend logic, and database structure were planned.

Step 3: Database Creation

SQLite database tables were created for storing books, members, and issue records.

Step 4: Frontend Development

The graphical user interface was developed using Tkinter. Forms, buttons, labels, frames, and tables were created.

Step 5: Backend Development

Python functions were developed to handle the operations of adding, viewing, issuing, and returning books.

Step 6: Database Integration

The Python backend was connected with the SQLite database using the sqlite3 module.

Step 7: Testing

Each module was tested individually. Books and members were added, books were issued and returned, and the records were checked through the view modules.

Step 8: Final Implementation

After successful testing, all modules were integrated into a single Library Management System application.

MODULES OF THE SYSTEM

The Library Management System consists of the following modules:

1. Dashboard

The dashboard provides an overview of the library. It displays the total number of books, total members, issued books, and returned books.

2. Add Book

This module allows the user to add new books to the library database.

3. View Books

This module displays all the books stored in the database.

4. Add Member

This module allows the user to register new library members.

5. View Members

This module displays all registered members.

6. Issue Book

This module allows a book to be issued to a registered member. The system checks the availability of the book before issuing it.

7. View Issued Books

This module displays all books that have been issued and have not yet been returned.

8. Return Book

This module allows the user to return an issued book. The available quantity of the book is automatically increased.

9. View Returned Books

This module displays the complete records of books that have been returned.

WORKING OF THE SYSTEM

The Library Management System works through a simple interaction between the user interface, Python backend, and SQLite database.

First, the user opens the application. The main dashboard is displayed along with the navigation menu.

When the user selects an operation, the corresponding form or table is displayed.

For adding a book, the user enters the Book ID, Title, Author, Category, and Quantity. After clicking the Add Book button, the Python backend validates the information and stores it in the Books table.

Similarly, members can be added through the Add Member module.

When a book is issued, the system checks whether the book is available and whether the member exists. If both conditions are satisfied, the transaction is stored in the Issues table and the book quantity is reduced.

When the book is returned, the return date is stored in the database and the quantity of the book is increased.

The View modules allow the user to check all records stored in the database.

1. Main Dashboard

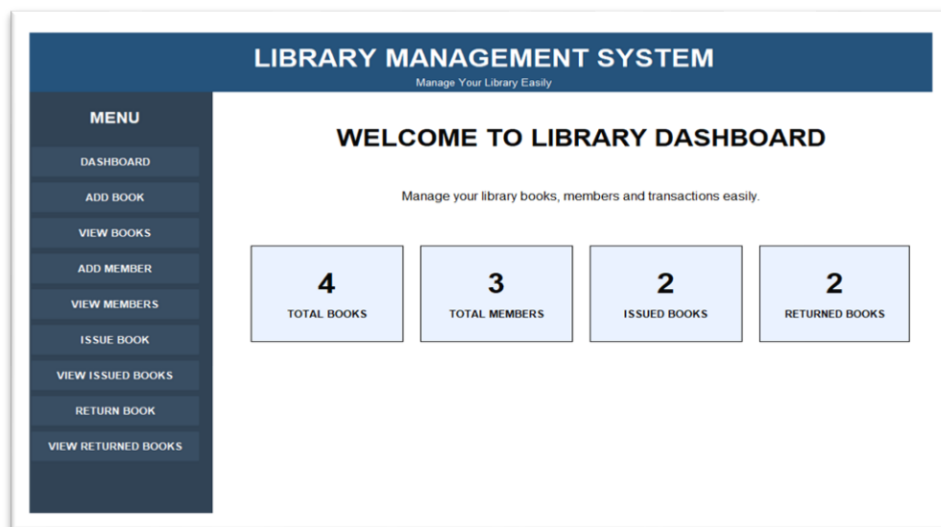


Figure 1: Main Dashboard

2. Add book

The screenshot shows the 'ADD NEW BOOK' form within the 'LIBRARY MANAGEMENT SYSTEM'. The system has a dark blue header with the title and tagline 'Manage Your Library Easily'. A left sidebar menu lists various options: DASHBOARD, ADD BOOK, VIEW BOOKS, ADD MEMBER, VIEW MEMBERS, ISSUE BOOK, VIEW ISSUED BOOKS, RETURN BOOK, and VIEW RETURNED BOOKS. The main form area contains input fields for Book ID, Book Title, Author, Category, and Quantity, followed by a green 'ADD BOOK' button.

Book ID	Book Title	Author	Category	Quantity

Figure 2: Add Book Interface

3.View Book

The screenshot shows the 'ALL BOOKS' view within the 'LIBRARY MANAGEMENT SYSTEM'. It features the same header and sidebar menu as Figure 2. The main area displays a table with the following data:

Book ID	Title	Author	Category	Quantity
101	python programming	r.sharma	programming	5
102	database management	K.Sharma	database	3
103	computer network	A.Kumar	networking	2
104	software engineering	R.Pressman	Software	2

Figure 3: View Book Interface

4.Add Member

The screenshot shows the 'ADD MEMBER' form within the 'LIBRARY MANAGEMENT SYSTEM' interface. The system title and tagline 'Manage Your Library Easily' are at the top. A dark blue sidebar menu on the left contains options: MENU, DASHBOARD, ADD BOOK, VIEW BOOKS, ADD MEMBER (highlighted), VIEW MEMBERS, ISSUE BOOK, VIEW ISSUED BOOKS, RETURN BOOK, and VIEW RETURNED BOOKS. The main content area is titled 'ADD MEMBER' and contains four input fields: Member ID, Name, Course, and Contact. A green 'ADD MEMBER' button is positioned below the fields.

Member ID	Name	Course	Contact
1101	radhika	machine learning using python	8765346***
1102	priya	computer science	784894***
1103	aditya	information technology	6428364***

Figure 4: Add Book Interface

5.View Member

The screenshot shows the 'ALL MEMBERS' view within the 'LIBRARY MANAGEMENT SYSTEM' interface. The system title and tagline 'Manage Your Library Easily' are at the top. The same dark blue sidebar menu is on the left. The main content area is titled 'ALL MEMBERS' and displays a table with member information.

Member ID	Name	Course	Contact
1101	radhika	machine learning using python	8765346***
1102	priya	computer science	784894***
1103	aditya	information technology	6428364***

Figure 5: View Member Interface

6.Issue Book

LIBRARY MANAGEMENT SYSTEM
Manage Your Library Easily

MENU

- DASHBOARD
- ADD BOOK
- VIEW BOOKS
- ADD MEMBER
- VIEW MEMBERS
- ISSUE BOOK
- VIEW ISSUED BOOKS
- RETURN BOOK
- VIEW RETURNED BOOKS

ISSUE BOOK

Book ID

Member ID

Issue Date (DD-MM-YYYY)

ISSUE BOOK

Figure 5: Issue Book Interface

7.View Issued Book

LIBRARY MANAGEMENT SYSTEM

Manage Your Library Easily

MENU

DASHBOARD

ADD BOOK

VIEW BOOKS

ADD MEMBER

VIEW MEMBERS

ISSUE BOOK

VIEW ISSUED BOOKS

RETURN BOOK

VIEW RETURNED BOOKS

ISSUED BOOKS

	Issue ID	Book ID	Member ID	Issue Date
3	102	1102	01-09-2026	
4	103	1102	01-09-2026	

Figure 6: View Issued Book Interface

7. Return Book

The screenshot shows the 'RETURN BOOK' interface. On the left is a dark blue sidebar menu with the following items: MENU, DASHBOARD, ADD BOOK, VIEW BOOKS, ADD MEMBER, VIEW MEMBERS, ISSUE BOOK, VIEW ISSUED BOOKS, RETURN BOOK (highlighted), and VIEW RETURNED BOOKS. The main content area has a blue header with 'LIBRARY MANAGEMENT SYSTEM' and the tagline 'Manage Your Library Easily'. Below the header, the title 'RETURN BOOK' is centered. There are two input fields: 'Issue ID' and 'Return Date (DD-MM-YYYY)'. A green button labeled 'RETURN BOOK' is positioned below the input fields.

Figure 7: Return Book Interface

8.View Return Book

The screenshot shows the 'RETURNED BOOKS' interface. On the left is a dark blue sidebar menu with the following items: MENU, DASHBOARD, ADD BOOK, VIEW BOOKS, ADD MEMBER, VIEW MEMBERS, ISSUE BOOK, VIEW ISSUED BOOKS, RETURN BOOK, and VIEW RETURNED BOOKS (highlighted). The main content area has a blue header with 'LIBRARY MANAGEMENT SYSTEM' and the tagline 'Manage Your Library Easily'. Below the header, the title 'RETURNED BOOKS' is centered. A table displays the returned books with the following data:

	Issue ID	Book ID	Member ID	Issue Date	Return Date
1		101	1101	31-08-2026	31-08-2026
2		101	1101	31-08-2026	01-09-026

Figure 8: View Return Book Interface

CONCLUSION

The Library Management System has been successfully developed using Python, Tkinter, and SQLite.

The project provides an efficient way to manage books, library members, book issuing, and book returning operations. The use of a graphical user interface makes the system easy to use and understand.

The frontend provides a clean and user-friendly environment, while the Python backend handles all the application logic and database operations. SQLite provides a simple and efficient method for storing records without requiring a separate database server.

The system successfully performs important operations such as adding books, viewing books, adding members, viewing members, issuing books, viewing issued books, returning books, and viewing returned books.

Overall, this project demonstrates the practical implementation of Python programming, GUI development, database management, and software development concepts. The Library Management System reduces manual work and provides an organized way to maintain library records.

FUTURE SCOPE

The Library Management System can be further improved by adding more advanced features in the future.

1. A login system can be added for administrators and library staff.
2. Student login functionality can be introduced.
3. A search feature can be added to search books by title, author, or category.
4. Book cover images can be added to make the interface more attractive.
5. A fine calculation system can be added for late book returns.
6. Book reservation functionality can be introduced.
7. Barcode or QR code scanning can be integrated.
8. Email or notification features can be added to remind members about return dates.
9. The SQLite database can be replaced with MySQL for handling larger amounts of data.
10. The project can be converted into a web-based or mobile application.
11. Reports can be generated automatically in PDF or Excel format.
12. Additional security features and user authentication can be implemented.

BIBLIOGRAPHY

The following resources were referred to during the development of this project:

1. Python Official Documentation – Python Programming Language.
2. Tkinter Documentation – Python Graphical User Interface Library.
3. SQLite Official Documentation – Database Management System.
4. Anaconda Documentation – Python Distribution and Development Environment.
5. Spyder Documentation – Python Integrated Development Environment.
6. Books and online learning resources related to Python programming, GUI development, and database management.